TRACE-CERTIFIED HYBRID LDPC DECODING: A SELL-STYLE OPERATIONAL SPECIFICATION

CARLOS RAMÍREZ OVALLE

Pontificia Universidad Javeriana, Cali, Colombia
e-mail address: carlosovalle@javerianacali.edu.co

Abstract. Hybrid decoding over the binary erasure channel combines peeling with a residual linear solve. We study independently checkable certificates for this classical algorithm using a specification inspired by subexponential linear logic. A delayed-propagation invariant connects asynchronous multiset rewriting to operational peeling: every rewrite execution emits a valid trace, and every valid trace has a corresponding derivation with an exact logged endpoint after propagation normalization. Success certificates provide a compatible codeword; failure certificates provide a linear infeasibility witness. We prove their soundness and existence, and distinguish maximum-likelihood completion from unique or exact recovery. The two persistent subexponential labels organize the specification but are not necessary for these operational rules; no adequacy with a focused logical calculus is claimed. Sparse trace checking and direct full-word checking have the same linear upper bound in graph size, while providing different trace guarantees. A reproducible implementation measures their costs separately, including corrupted inputs and malformed certificates. The companion Lean development and the implementation tests have explicitly distinct verification scopes.

1. Introduction

An untrusted decoder can return a proposed word together with evidence that a smaller checker validates. For a binary erasure channel (BEC), the final-word condition is particularly simple: the word must satisfy the parity checks and agree with all visible symbols. A direct syndrome check already establishes this property. Replaying a peeling trace adds a different guarantee: each declared local assignment was justified by a degree-one constraint. The question studied here is how to connect that trace guarantee to a precise logical-style specification while retaining simple certificates for the residual algebraic phase.

The decoding algorithm is classical. Peeling resolves degree-one checks; Gaussian elimination completes the remaining linear system [RU08, BM04]. Independent result checking is also established methodology [MMNS11, ABMR13, Nec97]. Our construction is a case study in connecting these ingredients. The delicate point is asynchronous state: after a variable is solved, some adjacent checks may still await its value while another variable is solved elsewhere. Identifying such an intermediate context with a fully updated operational

Key words and phrases: LDPC codes, binary erasure channel, certifying algorithms, subexponential linear logic, multiset rewriting, trace adequacy, Lean.

Preprint submitted to
Logical Methods in Computer Science

© C. Ramírez Ovalle
Creative Commons

state gives incorrect degrees and syndromes. Discarding log atoms from an exact reachability target creates a second, independent error. The semantics developed here accounts for both.

The mathematical development has three parts. First, a pending-incidence invariant proves soundness for arbitrary rewrite interleavings and completeness for each specified operational trace, with normalization and logged endpoints made explicit. Second, a residual-system bijection connects valid traces to success witnesses and classical left-kernel infeasibility witnesses. Third, an event-preserving translation shows that the two persistent labels can be merged for these rules. Thus SELL supplies a useful resource discipline without an unsupported necessity claim. These results concern the defined transition system, rather than proof search in an unformalized SELL calculus.

The artifact complements the mathematics with strict certificate validation, an independent direct-word checker, finite adversarial and exhaustive tests, and a controlled experiment using the same algebraic engine for the compared decoding pipelines. We report completion, exact recovery, uniqueness and corruption compatibility separately. The proof assistant's scope and the Python trust boundary are stated explicitly; similar-looking implementations are not assumed equivalent. The source repository is `https://github.com/cero1979/SELLTC`. Its current access restrictions and release requirements are recorded in the companion readiness report; this manuscript does not assert that a public archival release exists.

2. Coding and operational preliminaries

Let $H \in \mathbb{F}_2^{m\times n}$, with variable set $\mathsf{Var} = \{1,\dots,n\}$, check set $\mathsf{Chk} = \{1,\dots,m\}$, and Tanner edges $\mathcal{E} = \{(j,i) : H_{ji} = 1\}$. Write $\mathcal{N}(j) = \{i : H_{ji} = 1\}$ and $C = \{x \in \mathbb{F}_2^n : Hx = 0\}$. Vectors are columns throughout. The matrix need not have full row rank, and empty rows, empty columns, $m = 0$ and $n = 0$ are allowed. Sparsity is needed for the implementation bounds, not for the algebraic theorems. LDPC decoding and its standard iterative and algebraic ingredients are classical [Gal62, RU08, LMSS01, BM04].

A received word is $y \in \{0,1,\star\}^n$. A partial assignment $K : \mathsf{Var} \rightharpoonup \mathbb{F}_2$ has unknown set $U = \mathsf{Var} \setminus \operatorname{dom} K$. A full word x extends K if $x_i = K(i)$ whenever $K(i)$ is defined. Initially $K_0(i) = y_i$ at each nonerased coordinate. For an assignment K, define

$$s_K(j) = \sum_{i\in\mathcal{N}(j)\cap\operatorname{dom} K} K(i), \qquad d_K(j) = |\mathcal{N}(j)\cap U|. \tag{2.1}$$

All sums and matrix products are in $\mathbb{F}_2$, unless explicitly stated otherwise. Thus addition is XOR. A coherent operational state $\sigma = (K, U, s, d)$ has $s = s_K$ and $d = d_K$.

Definition 2.1 (Operational peeling and trace)**.** A triple $a = (j, i, b)$ is valid at σ if $\mathcal{N}(j)\cap U = \{i\}$ and $b = s(j)$. Its successor extends K by $K(i) = b$, removes i from U, and, for every ℓ adjacent to i, sets $s(\ell) \leftarrow s(\ell) + b$ and $d(\ell) \leftarrow d(\ell) - 1$. Other entries remain unchanged. A trace $T = (a_1,\dots,a_t)$ is valid from y when every triple is valid at the preceding state, starting at K_0. Traces may be empty or stop before peeling is maximal. Every valid trace has $t \le n$ because it resolves a different unknown coordinate at every step.

Lemma 2.2 (State meaning and preservation of completions)**.** *Every state reached by a valid trace satisfies* (2.1)*. Furthermore, every $x \in C$ agreeing with the visible coordinates of y extends its reached assignment K.*

Proof. Both state identities hold initially by definition. In a valid step, i was unknown, so adding it to $\operatorname{dom} K$ adds exactly b to the known sum at each adjacent check and removes exactly one term from the corresponding unknown-neighbor count. Nonadjacent checks are unaffected. This proves the identities inductively.

Initially the assertion about x is its assumed agreement with y. Suppose x extends the current K, and the next step is (j, i, b). Every neighbor of j except i is already known. Since the j-th parity of x is zero,

$$x_i = \sum_{h \in \mathcal{N}(j) \setminus \{i\}} x_h = \sum_{h \in \mathcal{N}(j) \cap \operatorname{dom} K} K(h) = s(j) = b.$$

Thus x extends the enlarged assignment too. Induction proves the claim for the entire trace, including inputs that have no completion (for which the assertion is vacuous). □

A nonempty $S \subseteq \mathsf{Var}$ is a *stopping set* when every check has either zero or at least two neighbors in S [DPT+02].

Proposition 2.3 (Stopping sets)**.** *A maximal peeling run leaves a nonempty unknown set precisely when that remaining set is a stopping set. Peeling removes all initial erasures if and only if the initial erasure set contains no stopping set. Here "removes" is a structural statement, not acceptance of a codeword.*

Proof. A step exists exactly when some check has one unknown neighbor. Consequently a nonempty residual has no possible step exactly when it is a stopping set. Now let S be a stopping subset of the initial unknown set. If peeling removed a member of S, consider the first such removal. At its witnessing check all members of S are still unknown; that check touches S and hence has at least two unknown neighbors, contradicting degree one. Thus S survives every run. Conversely, a nonempty residual of a maximal run is itself a stopping subset of the initial erasure set, by the first part. These implications establish the characterization independently of the choice of eligible checks. A degree-zero row with syndrome one may still rule out any completion; it does not change the structural argument. □

3. Residual algebra and certificate checking

For the final unknown set U of a valid trace, order its coordinates in increasing order, let H_U be the corresponding column restriction of H, and put $r = (s(j))_{j \in \mathsf{Chk}}$. A residual assignment $\rho \in \mathbb{F}_2^{|U|}$ fills those coordinates, and $K \cup \rho$ denotes the resulting full word.

Lemma 3.1 (Residual equivalence)**.** *For a valid trace from y, the map $\rho \mapsto K \cup \rho$ is a bijection between solutions of $H_U \rho = r$ and codewords agreeing with all visible coordinates of y.*

Proof. For a word extending K, its j-th parity is $s(j) + \sum_{i \in \mathcal{N}(j) \cap U} x_i$, by Lemma 2.2. This parity is zero exactly when the j-th equation of $H_U x_U = r$ holds. Hence every residual solution produces a compatible codeword. In the other direction, Lemma 2.2 says that every compatible codeword extends K; its restriction to U therefore solves the residual system. Restriction and filling are inverse functions. □

Definition 3.2 (Certificate formats and independent checker)**.** A success certificate consists of the tag success, a trace T, and a binary witness ρ of length $|U|$ after replay. A failure certificate consists of the tag failure, a trace T, and a witness $\omega \in \mathbb{F}_2^m$. The checker first rejects a trace longer than n (or stops a streaming trace after its $(n+1)$-st entry). It then validates the tag, triple structure, integer index bounds, binary values and witness lengths. It initializes the coherent state from (H, y) and replays every triple according to Definition 2.1; it rejects at the first invalid step. For success it accepts exactly when $H_U \rho = r$, returning $K \cup \rho$. For failure it accepts exactly when

$$\omega^\top H_U = 0, \qquad \omega^\top r = 1. \tag{3.1}$$

The checker uses only the supplied instance and certificate; it does not invoke the decoder. All rows are checked, including zero-degree rows. A schema-valid partial trace is allowed in both branches.

A hybrid producer runs maximal peeling, constructs the residual, and performs exact Gaussian elimination over $\mathbb{F}_2$. A solution produces a success certificate; an inconsistent row produces a failure certificate as described below. When U is empty, the residual still has m equations: success requires $r = 0$. Removing all erasures does not justify skipping this check.

Theorem 3.3 (Success soundness)**.** *If the checker accepts a success certificate and returns x, then $x \in \mathbb{F}_2^n$, $Hx = 0$, and $x_i = y_i$ at every nonerased coordinate.*

Proof. The validated binary witness and assignment K define a binary word. Replay only assigns initially unknown coordinates, so K extends K_0 and channel agreement holds. The accepted residual equation and Lemma 3.1 establish $Hx = 0$. □

Lemma 3.4 (Finite-field alternative)**.** *For $A \in \mathbb{F}_2^{m \times q}$ and $r \in \mathbb{F}_2^m$, exactly one of the following exists: a solution z of $Az = r$, or a vector ω with $\omega^\top A = 0$ and $\omega^\top r = 1$.*

Proof. They cannot both exist, since then $1 = \omega^\top r = \omega^\top A z = 0$. Suppose there is no solution. Let W be the span of the columns of A. Then $r \notin W$. Choose a basis of W; appending r gives a linearly independent family, which extends to a basis of the finite space $\mathbb{F}_2^m$. Define a linear functional f on this basis to be zero on the basis of W, one on r, and zero on all additionally chosen basis vectors. Unique basis expansion defines f on the whole space and proves its linearity. If e_j are the coordinate basis vectors and $\omega_j = f(e_j)$, then for every vector v, $f(v) = \sum_j v_j f(e_j) = \omega^\top v$. Therefore f annihilates every column of A, while $f(r) = 1$, giving the required witness. The argument also covers $q = 0$. If $m = 0$, the zero vector of $\mathbb{F}_2^q$ is always a solution and the inconsistent case cannot arise. □

Proposition 3.5 (Constructing a failure witness)**.** *Suppose elementary row operations transform $[A \mid r]$ to $[R \mid c] = M[A \mid r]$, where $M \in \mathbb{F}_2^{m \times m}$ records the same operations on the identity matrix. If R has a zero row h with $c_h = 1$, then $\omega^\top = e_h^\top M$ satisfies the failure conditions. Row reduction of an inconsistent system always finds such a row.*

Proof. Taking row h of the displayed equality gives $(e_h^\top M)A = R_{h,*} = 0$ and $(e_h^\top M)r = c_h = 1$. Row swaps and adding one row to another are invertible over $\mathbb{F}_2$, so they preserve solvability. In echelon form, if every zero coefficient row has right-hand side zero, set all free variables to zero and solve the pivot equations from bottom to top. This constructs a solution: each pivot is one, later pivot variables are already assigned, and zero rows impose no further condition. Consequently an inconsistent echelon system must have a zero

coefficient row with right-hand side one. Redundant rows and zero-column matrices cause no exception. □

Theorem 3.6 (Failure soundness and certificate completeness)**.** *An accepted failure certificate excludes every codeword agreeing with y. Conversely, for every (H, y) and every valid finite trace T, there is an accepted certificate of exactly one status: success if a compatible codeword exists, and failure otherwise.*

Proof. For soundness, acceptance supplies (3.1). By Lemma 3.1, any compatible codeword would supply a residual solution. This contradicts Lemma 3.4. For completeness, if a compatible codeword exists, its restriction to U supplies a success witness by the residual bijection. Otherwise the residual is inconsistent and the finite-field alternative supplies a failure witness. Both witnesses pass the specified checks, and the two statuses cannot both be accepted by the same alternative. This is an existence statement for a suitable witness, not a claim that every alleged witness is accepted. □

Proposition 3.7 (Multiplicity and exact recovery)**.** *If $H_U\rho = r$ is consistent and has rank a, it has exactly $2^{|U|-a}$ solutions. Hence the compatible completion is unique if and only if $|U|-a = 0$. Under a clean erasure channel the transmitted word is one of these completions; acceptance alone does not identify it when more than one completion exists.*

Proof. For one solution ρ_0, every solution is uniquely of the form $\rho_0 + v$ with $v \in \ker H_U$, because $H_U(\rho + \rho_0) = r + r = 0$. Rank–nullity gives dimension $|U| - a$ for this kernel. A basis of a d-dimensional binary vector space gives a bijection with $\mathbb{F}_2^d$, which has 2^d elements. Apply the residual bijection. On the clean channel the transmitted codeword agrees with the visible symbols, proving the last assertion. □

4. Likelihood, examples and trust boundary

For erasure probability $\epsilon \in [0, 1]$, define the memoryless BEC transition probability by

$$W_\epsilon(y \mid x) = \prod_{i=1}^{n} w_\epsilon(y_i \mid x_i), \qquad w_\epsilon(a \mid b) = \begin{cases} \epsilon, & a = \star, \\ 1 - \epsilon, & a = b, \\ 0, & a \in \mathbb{F}_2 \text{ and } a \neq b. \end{cases}$$

Maximum likelihood means maximizing this quantity over $x \in C$; it does not impose a prior distribution on codewords.

Proposition 4.1 (BEC maximum likelihood)**.** *For every $\epsilon \in [0, 1]$, a compatible codeword maximizes $W_\epsilon(y \mid x)$ over C. If $0 < \epsilon < 1$ and a compatible codeword exists, the maximizers are exactly the compatible codewords.*

Proof. Let u and $k = n - u$ be the numbers of erased and visible coordinates. Every compatible word has likelihood $c = \epsilon^u(1-\epsilon)^k \geq 0$; every incompatible word has a zero factor and therefore likelihood zero. Thus no candidate has likelihood greater than a compatible word. For interior ϵ, $c > 0$, so the incompatible candidates are not maximizers. The product definition handles the endpoints, with an empty product equal to one. An observation impossible at an endpoint may give all candidates likelihood zero; the first assertion still holds, but the second need not. No posterior conditional on such a zero-probability event is asserted. □

Example 4.2 (Peeling, stopping, ambiguity and impossibility)**.** All indices in these calculations are one-based; the software uses zero-based indices.

(a) For $H = \left(\begin{smallmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{smallmatrix}\right)$ and $y = (0, \star, \star)$, initially $s = (0, 0)$ and $d = (1, 2)$. The step $(1, 2, 0)$ changes these to $s = (0, 0)$, $d = (0, 1)$. The step $(2, 3, 0)$ leaves $U = \varnothing$ and $r = (0, 0)$, so the empty residual assignment certifies $x = (0, 0, 0)$.

(b) For $H = (1\ 1)$ and $y = (\star, \star)$, the unknown set is a stopping set and the empty trace is maximal. The residual equation $\rho_1 + \rho_2 = 0$ has solutions $(0, 0)$ and $(1, 1)$, rank one and nullity one. Each is an ML completion, and neither is selected by likelihood alone.

(c) For $H = \left(\begin{smallmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{smallmatrix}\right)$, $y = (\star, 0, 1)$ and $T = \varnothing$, the residual is $A = (1, 1)^\top$, $r = (0, 1)^\top$. The witness $\omega = (1, 1)^\top$ gives $\omega^\top A = 0$ and $\omega^\top r = 1$. It certifies impossibility before any peeling. Alternatively resolving $x_1 = 0$ from row one leaves no residual variables and right-hand side $(0, 1)$; $\omega = (0, 1)^\top$ then certifies the remaining degree-zero contradiction.

(d) For $H = (1\ 1)$, a transmission of $(0, 0)$ followed by one visible flip and one erasure can yield $y = (1, \star)$. Peeling validly returns $(1, 1)$, which passes both parity and channel checks. The certificate is valid, but the transmitted word was not recovered. Corruption need not make the observed constraints inconsistent.

The trusted instance is the supplied pair (H, y), including its validated graph representation. The mathematical checker assumes exact binary arithmetic; an executable deployment also trusts its parser, checker implementation, interpreter/compiler and runtime. The decoder and its residual solver need not be trusted. A Lean proof of a mathematical function does not by itself verify an independently written Python program or its execution environment.

An accepted trace certifies that the declared sequence is permitted by the specification. It does not attest that an untrusted producer actually followed this sequence internally. A valid partial trace, or a permutation of independent steps that remains valid, must be accepted. Malformed certificates, incompatible words and invalid steps must be rejected. Rejection of a malformed failure witness alone does not establish that a completion exists.

If the supplied matrix is corrupted, acceptance proves only the constraints of that supplied matrix, not those of an intended external code. Authenticating the matrix or channel observations is outside this model. On inputs containing visible flips, compatibility is an algebraic property; the BEC likelihood theorem is not a guarantee about the physical channel that generated them.

5. Representation, verification cost and certificate size

Assume a validated graph stored as check-to-variable and variable-to-check adjacency lists with no duplicate incidences. Use a word-RAM with words large enough for indices and counters, constant-time binary arithmetic and array access. Graph validation and construction from a dense input matrix require $\Theta(mn)$ entry inspections; they are separate from the following per-input bounds. Storing a matrix densely does not become a sparse algorithm merely by calling a pivot heuristic "sparse."

Proposition 5.1 (Sparse checking upper bound)**.** *A success or failure certificate can be checked in $O(n + m + |\mathcal{E}|)$ word operations and $O(n + m + |\mathcal{E}|)$ space, including graph storage. Direct verification of a binary full word against H and y has the same asymptotic upper bound and requires no trace.*

Proof. Reading y and initializing variable status costs $O(n)$; initializing check arrays costs $O(m)$. Visiting each incidence once computes syndromes, residual degrees and the XOR of the indices of each check's unknown neighbors, at cost $O(|\mathcal{E}|)$. When a residual degree is one, this last XOR identifies its unique unknown index in constant time, even if that index is zero. The checker validates each triple using these arrays, then visits the adjacency list of its newly assigned variable to update the three check arrays. No variable is assigned twice, so the total number of such incidence visits is at most $|\mathcal{E}|$, and there are at most n valid triples. An excessive trace is rejected after at most $n+1$ inspected entries; malformed dimensions are rejected before witness processing.

For success, fill the remaining coordinates from the witness and accumulate their contributions across the unknown incidences, checking all m residual equations. For failure, allocate one binary accumulator per unknown variable, and visit each residual incidence (j, i) to add ω_j to its accumulator; separately compute $\sum_j \omega_j r_j$. These procedures cost $O(n + m + |\mathcal{E}|)$. The arrays have $O(n + m)$ entries and the graph has $O(|\mathcal{E}|)$ incidences. Direct checking reads the candidate and y and computes each parity by a single pass over the check adjacency lists. □

The bound simplifies to $O(n + |\mathcal{E}|)$ only under an assumption such as $m = O(n + |\mathcal{E}|)$. It is an upper bound for this representation, not an optimality theorem for every possible certificate format or preprocessed matrix. The bit cost of reading indices is not constant in a bit-complexity model. A parser that first materializes an arbitrarily long hostile input additionally pays for those bytes; the graph bound assumes a parsed certificate of bounded format or a streaming parser that enforces these limits.

With $t = |T|$, an index-encoded trace occupies $O(t(1 + \log(m + 1) + \log(n + 1)))$ bits, plus a length/tag description. A success witness adds $|U|$ bits in canonical variable order; a failure witness adds m bits. For $m = O(n)$ and $t \leq n$, these are $O(n \log(n + 1))$ and $O(n \log(n + 1) + m)$ respectively. A direct success witness is simply an n-bit word. It proves parity and visible-coordinate agreement without proving a declared peeling trace. Consequently trace replay purchases an additional explanatory property, not a stronger final-word validity guarantee. For an empty trace, a failure witness for the initial erasure system already certifies impossibility without any peeling history.

Certificate *construction* has a different cost. A straightforward dense elimination of an m by q residual, with m extra witness columns, takes $O(q + m(q + m)(1 + \min(m, q)))$ elementary bit operations, including initialization, row-array updates and pivot searches; its storage is $O(q+m(q+m))$ bits apart from indexing, including the returned solution. Packed machine-word rows can improve constants and word-operation bounds. Peeling may reduce q, but no universal speedup over GE-only follows, and recording the row transformations can itself be expensive. Our comparisons use the same algebraic engine to keep that distinction observable.

6. SELL-style rewriting and asynchronous adequacy

We specify peeling by two rules with persistent knowledge and linear residual resources. The operational semantics in this section is multiset rewriting. The theorem relates its executions to operational peeling, including executions that interleave solving with unfinished propagation. A logical reading of the clauses motivates the resource discipline; no adequacy with a SELL sequent calculus or its focused variant is assumed.

6.1. **Resources, clauses, and execution labels.** Fix a binary parity-check matrix H and write $\mathcal{N}(j) = \{i \in \mathsf{Var} : H_{j,i} = 1\}$. A context is $M = (G, P, L)$: $G = H$ is fixed static information, P is a multiset of pairs (i, b) representing persistent facts $\mathsf{val}(i, b)$, and L is a multiset of linear atoms

$$\mathsf{unk}(i), \quad \mathsf{pend}(j, i), \quad \mathsf{rem}(j, k, p), \quad \mathsf{step}(j, i, b).$$

Here $i \in \mathsf{Var}$, $j \in \mathsf{Chk}$, $b, p \in \mathbb{F}_2$, and $k \in \mathbb{N}$. A pending incidence is an edge whose variable has not yet been incorporated into that check's stored residual. Its variable may already be known. This distinction is necessary for asynchronous executions.

The two rules below act inside an arbitrary unchanged multiset context. A premise in the persistent zone is read without being removed. Every displayed linear premise is removed once, and every displayed conclusion is inserted once.

$$\textbf{Propagate:} \quad !^K\mathsf{val}(i, b) \otimes \mathsf{pend}(j, i) \otimes \mathsf{rem}(j, k+1, p) \multimap \mathsf{rem}(j, k, p \oplus b),$$

$$\textbf{Solve:} \quad \mathsf{rem}(j, 1, p) \otimes \mathsf{pend}(j, i) \otimes \mathsf{unk}(i) \multimap !^K\mathsf{val}(i, p) \otimes \mathsf{step}(j, i, p) \otimes \mathsf{rem}(j, 0, 0).$$

The successor form of the propagation degree excludes underflow. There are no rules that remove log atoms or create pending incidences. The matrix is not read by either rule: the correspondence between pending incidences and actual edges is established by the invariant below.

A rewrite event has label τ for Propagate and label (j, i, p) for Solve. An execution is a finite sequence of contexts and labelled events, with each event justified by the applicable rule. Its trace is the ordered list of its non-τ labels. We write

$$M \overset{T}{\Longrightarrow}{}^* M'$$

when an execution from M to M' has exactly trace T. The empty execution has trace $[\,]$; concatenation concatenates traces. Thus the trace is part of the execution data, not an arbitrary ordering of the final log multiset. In a proof assistant one may use computational derivation objects or a proposition indexed explicitly by T; endpoint reachability alone does not retain execution order.

For a trace T, let $\mathrm{Log}(T)$ be the multiset containing one $\mathsf{step}(j, i, b)$ for each occurrence of (j, i, b) in T. Write $\uplus$ for multiset sum, so multiplicities are always retained.

6.2. **An invariant for delayed propagation.**

Definition 6.1 (Admissible asynchronous context)**.** Let $K : \mathsf{Var} \rightharpoonup \mathbb{F}_2$ be a partial function and $U = \mathsf{Var} \setminus \mathrm{dom}(K)$. For each j, let Q_j be a finite set and $p_j \in \mathbb{F}_2$. A context is admissible with data (K, Q, p, T) if $G = H$, its persistent zone is exactly

$$P = \{\{(i, K(i)) : i \in \mathrm{dom}(K)\}\},$$

and its linear zone is exactly

$$\begin{aligned} L = {} & \{\{\mathsf{unk}(i) : i \in U\}\} \uplus \{\{\mathsf{pend}(j, i) : j \in \mathsf{Chk},\ i \in Q_j\}\} \\ & \uplus \{\{\mathsf{rem}(j, |Q_j|, p_j) : j \in \mathsf{Chk}\}\} \uplus \mathrm{Log}(T), \end{aligned}$$

where

$$\mathcal{N}(j) \cap U \ \subseteq \ Q_j \ \subseteq \ \mathcal{N}(j), \qquad p_j = \bigoplus_{i \in \mathcal{N}(j) \setminus Q_j} K(i). \tag{6.1}$$

Every multiset comprehension above contributes each indicated atom exactly once. The values in the XOR are defined because the first inclusion implies $\mathcal{N}(j) \setminus Q_j \subseteq \mathrm{dom}(K)$.

In particular, each variable has exactly one unknown token or exactly one persistent value; conflicting values are excluded. Each row has exactly one residual token, and pending incidences have multiplicity one. Membership-based uniqueness would not suffice: two copies of the same residual atom have the same parameters, but erasing one copy leaves the other behind.

Definition 6.2 (Operational projection and normalized encoding)**.** The operational projection of an admissible context is $\pi(M) = (K, U, s, d)$, where

$$d(j) = |\mathcal{N}(j) \cap U|, \qquad s(j) = \bigoplus_{i \in \mathcal{N}(j) \cap \mathrm{dom}(K)} K(i).$$

Let $D_j = Q_j \cap \mathrm{dom}(K)$ be its delayed known incidences. The context is *propagation-normalized* if every D_j is empty. For an operational state σ satisfying the degree and syndrome invariants, $\mathrm{Enc}_H(\sigma, T)$ is the admissible context with $Q_j = \mathcal{N}(j) \cap U$, $p_j = s(j)$, and log multiset $\mathrm{Log}(T)$.

The inclusion in (6.1) gives the disjoint union $Q_j = (\mathcal{N}(j) \cap U) \dot\cup D_j$. Consequently

$$|Q_j| = d(j) + |D_j|, \qquad s(j) = p_j \oplus \bigoplus_{i \in D_j} K(i). \tag{6.2}$$

These equations state precisely how the stored residual differs from the operational residual. In a normalized context they agree, and $M = \mathrm{Enc}_H(\pi(M), T)$. In an intermediate context the equalities $|Q_j| = d(j)$ and $p_j = s(j)$ need not hold.

Example 6.3 (Two solves before propagation finishes)**.** Let

$$H = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad y = (\star, \star).$$

Initially $Q_1 = \{1\}$, $Q_2 = \{2\}$, $Q_3 = \{1, 2\}$, and all stored syndromes are zero. Solve $(1, 1, 0)$ and then immediately solve $(2, 2, 0)$. Both rewrites are enabled. The resulting assignment knows both variables, but row 3 still contains $\mathsf{rem}(3, 2, 0)$ and both pending incidences. Its operational degree is zero and its delayed set is $D_3 = \{1, 2\}$. Either order of the two remaining propagations yields $\mathsf{rem}(3, 0, 0)$ and leaves the trace $[(1, 1, 0), (2, 2, 0)]$ unchanged. A proof that assumes all propagations follow their originating solve immediately would not cover this execution.

Lemma 6.4 (Initialization)**.** *Let σ_0 be the initial peeling state for y. Then $\mathrm{Enc}_H(\sigma_0, [\,])$ is admissible and normalized.*

Proof. Initialization sets K to the visible coordinates of y, $d(j) = |\mathcal{N}(j) \cap U|$, and $s(j)$ to the XOR of the known neighbors. With $Q_j = \mathcal{N}(j) \cap U$, its complement inside $\mathcal{N}(j)$ is precisely the known-neighbor set, so (6.1) holds with $p_j = s(j)$. The encoding contains each required atom exactly once and no log atoms. Also $Q_j \cap \mathrm{dom}(K) = \emptyset$. □

Lemma 6.5 (Propagation preserves the invariant and stutters)**.** *Suppose M is admissible with data (K, Q, p, T) and a Propagate event at (j, i) with value b yields M'. Then M' is admissible with the same K and T, and $\pi(M') = \pi(M)$.*

Proof. The premises and uniqueness of the persistent value imply $i \in Q_j \cap \mathrm{dom}(K)$ and $b = K(i)$. Replace Q_j by $Q_j \setminus \{i\}$ and p_j by $p_j \oplus b$; leave all other rows unchanged. No

unknown incidence is removed, so both inclusions in (6.1) remain true. The set $\mathcal{N}(j) \setminus Q_j$ gains exactly i, whose value is b; hence its XOR changes by precisely b. Since $i \in Q_j$, its cardinality decreases by one, as required by the new residual token. The unique pending and residual tokens are replaced exactly as specified, and no other token or log changes. The operational projection depends on H and K, both unchanged. □

Lemma 6.6 (Solve preserves the invariant and simulates peeling)**.** *Suppose M is admissible with data (K, Q, p, T) and Solve (j, i, b) yields M'. Then (j, i, b) is a valid operational peeling step from $\pi(M)$. Moreover M' is admissible with trace $T +\!\!+ [(j, i, b)]$, and $\pi(M')$ is precisely the operational state after that step.*

Proof. The residual token gives $|Q_j| = 1$ and $p_j = b$; the pending token gives $i \in Q_j$; and the unknown token gives $i \in U$. Thus $Q_j = \{i\}$. By (6.1), i is a genuine neighbor and every unknown neighbor of j lies in $\{i\}$. Therefore $\mathcal{N}(j) \cap U = \{i\}$. Every other neighbor is known and outside Q_j, so

$$b = p_j = \bigoplus_{v \in \mathcal{N}(j) \setminus \{i\}} K(v) = s(j).$$

This is exactly the operational validity condition. No assumption is needed about delayed incidences at other checks.

Set $K' = K \cup \{i \mapsto b\}$ and $U' = U \setminus \{i\}$. At row j, put $Q'_j = \emptyset$ and $p'_j = 0$. Its newly known-neighbor XOR is $p_j \oplus b = b \oplus b = 0$, agreeing with the rule. At any other row ℓ adjacent to i, the old inclusion $\mathcal{N}(\ell) \cap U \subseteq Q_\ell$ ensures $i \in Q_\ell$. It remains pending, so assigning its value does not change the XOR over $\mathcal{N}(\ell) \setminus Q_\ell$. Retain Q_ℓ, p_ℓ. At nonadjacent rows the sets and sums do not change. The pending inclusions hold with U' at every row.

Exactly one unknown token disappears and exactly one previously absent persistent assignment is inserted. The forcing row's residual and pending tokens are replaced as above; all other residual resources are retained. One log atom is added. Hence the complete admissibility conditions hold. Finally, fixing i removes one unknown neighbor and adds b to the known-neighbor XOR at precisely the rows adjacent to i. This is the operational update of d and s, and proves the projection claim. □

6.3. **Normalization and exact trace adequacy.**

Lemma 6.7 (Propagation normalization and commutation)**.** *Every admissible context M with data (K, Q, p, T) has a unique propagation-normal form, denoted* $\mathrm{NF}(M)$*, and*

$$\mathrm{NF}(M) = \mathrm{Enc}_H(\pi(M), T).$$

Every maximal propagation-only execution has exactly $\sum_j |D_j|$ events. Two distinct enabled propagations commute.

Proof. A propagation is enabled exactly at a delayed incidence (j, i) with $i \in D_j$: its persistent value and pending token are present, and $|Q_j| \geq 1$. By Lemma 6.5, such an event leaves K fixed and removes exactly this incidence from D_j. Thus the natural number $\sum_j |D_j|$ decreases by one at each event; it is zero exactly when no propagation is enabled. Every maximal propagation-only execution therefore terminates in the stated number of events.

At termination each pending set is $\mathcal{N}(j) \cap U$ by the two inclusions in (6.1). The invariant determines its syndrome as the XOR of all known neighbors. Assignments, unknown tokens,

static information, and logs are unchanged. Exact multiplicities now force the terminal context to be $\mathrm{Enc}_H(\pi(M), T)$, proving uniqueness.

For commutation, distinct enabled propagations at different rows modify disjoint pending and residual tokens and read persistent values only. At the same row they concern distinct pending variables, so the initial pending degree is at least two and either remains enabled after the other. Both orders remove those two incidences, decrease the degree by two, and replace p by $p \oplus b \oplus c$. Associativity and commutativity of XOR make the results equal. The same incidence cannot be consumed twice because its pending token has multiplicity one. □

Proposition 6.8 (Termination of rewriting)**.** *Every rewrite execution starting from an admissible context is finite. Its number of events is at most the initial number $\sum_j |Q_j|$ of pending incidences, and its number of Solve events is at most the initial $|U|$.*

Proof. Each rule consumes exactly one pending token and creates none. The preceding lemmas preserve the invariant, so $\sum_j |Q_j|$ decreases by one at every event and cannot become negative. Each Solve also consumes exactly one unknown token; neither rule creates one. □

The confluence statement above concerns propagation with a fixed assignment. It does not assert confluence of arbitrary Solve choices. On an inconsistent input two different degree-one checks may force different values of the same variable; both resulting traces can be locally valid, with inconsistency remaining in another row. Certificate checking still tests the entire residual system.

Example 6.9 (Why the endpoint includes a log)**.** For $H = [1]$ and $y = (\star)$, the valid step $(1, 1, 0)$ transforms

$$P = \emptyset, \quad L = \{\mathsf{unk}(1), \mathsf{pend}(1, 1), \mathsf{rem}(1, 1, 0)\}$$

into $P' = \{(1, 0)\}$ and $L' = \{\mathsf{rem}(1, 0, 0), \mathsf{step}(1, 1, 0)\}$. An encoding of only the final peeling state would omit the step atom. No further rule can erase it. Thus exact reachability of a log-free final encoding is false even in this one-variable example. The logged encoding Enc_H makes the correct endpoint explicit.

Theorem 6.10 (Adequacy with exact trace labels)**.** *Let σ_0 be the initial state for (H, y), and let $M_0 = \mathrm{Enc}_H(\sigma_0, [\,])$.*

(1) Soundness for arbitrary interleavings. *If $M_0 \overset{T}{\Longrightarrow}{}^{*} M$, then T is a valid operational peeling trace. Writing σ_T for its replayed state, M is admissible, $\pi(M) = \sigma_T$, its log is exactly $\mathrm{Log}(T)$, and*
$$\mathrm{NF}(M) = \mathrm{Enc}_H(\sigma_T, T).$$
(2) Completeness for each specified trace. *For every valid operational peeling trace T from σ_0,*
$$M_0 \overset{T}{\Longrightarrow}{}^{*} \mathrm{Enc}_H(\sigma_T, T).$$

These statements include empty and partial traces; neither requires that peeling has already stalled.

Proof. For soundness, induct on the events of the given execution, maintaining admissibility, exact log equality, and equality between the operational projection and replay of the trace prefix. The empty execution satisfies these assertions by Lemma 6.4. A Propagate event

preserves the invariant and projection by Lemma 6.5; its label τ does not extend the trace or the log. A Solve event is a valid operational step by Lemma 6.6, and that lemma supplies precisely the updated projection and appended trace/log entry. This proves all assertions about the endpoint for every finite execution, including those that end with unfinished propagations. Applying Lemma 6.7 gives the stated normalized endpoint.

For completeness, induct on the length of the requested trace, maintaining an execution to its exact logged normalized encoding. The empty trace is realized by the empty execution. Suppose the prefix T is realized at $\mathrm{Enc}_H(\sigma_T, T)$ and its next valid step is (j, i, b). Since $\mathcal{N}(j) \cap U = \{i\}$ and $s(j) = b$, that encoding contains $\mathsf{rem}(j, 1, b)$, $\mathsf{pend}(j, i)$, and $\mathsf{unk}(i)$. Apply Solve. For every other adjacent row ℓ, the incidence (ℓ, i) was pending because i was unknown; it is now a delayed known incidence. No other delayed incidence is introduced, since the starting context was normalized. Propagate these incidences in any order. Each remains enabled until it is consumed; commutation and normalization follow from Lemma 6.7. By Lemma 6.6 the projection is the next operational state, and propagation does not change it. The final context is therefore its exact normalized encoding with log $\mathrm{Log}(T +\!\!+ [(j, i, b)])$. The constructed block contributes exactly the requested triple, because its remaining events are all τ. Concatenating it with the prefix execution establishes the induction step. □

Proposition 6.11 (Maximal executions and stopping)**.** *The endpoint of a maximal rewrite execution from M_0 is normalized. Its projected unknown set is empty or is a nonempty stopping set.*

Proof. If a delayed known incidence remained, a propagation would be enabled, contradicting maximality. In a normalized context $Q_j = \mathcal{N}(j) \cap U$. If any check had exactly one unknown neighbor, its residual, pending, and unknown tokens would enable Solve. Thus no residual degree is one. When $U \neq \emptyset$, every check adjacent to U has degree at least two, which is precisely the stopping-set condition. □

This proposition concerns exhaustion of local peeling. In particular, $U = \emptyset$ does not imply that all parity equations hold: a row may have residual syndrome one. Both empty and nonempty residuals must undergo the algebraic certificate checks.

6.4. **What the subexponential labels contribute.** Following the resource distinction of linear logic and its subexponential extension [Gir87, NM09], one possible logical vocabulary uses the subexponential signature $\Sigma = (\{S, K\}, \preceq, \{S, K\})$, where $\preceq$ is the discrete preorder and both labels admit weakening and contraction. The S zone contains static edge facts and reusable program clauses; the K zone contains learned value facts. Linear atoms have no such prefix. The displayed clauses can be read as first-order multiplicative formulas, with finite variable and check sorts, natural-number degree indices, and $\mathbb{F}_2$-valued indices. Equivalently, use their ground instances, interpreting successor and XOR in these index domains. This specifies the intended clause syntax and structural permissions. In this paper the executable meaning is exactly the two operational rules already defined. Structural proof rules are not extra transitions of that operational system.

The two unbounded labels distinguish static data from learned information, but their distinction is not necessary for the operational result. The following statement identifies the precise scope of that observation.

Proposition 6.12 (Erasing persistent-zone labels preserves these executions)**.** *Consider the two-rule system above, with static edges represented by persistent* edge *facts. Replace both* $!^S$ *and* $!^K$ *by one ordinary* $!$*, retaining the atom predicates and event labels. On contexts in which edge facts occupy* S *and value facts occupy* K*, merging the two persistent zones is an event-preserving strong bisimulation. The same event-labelled transition system can be represented by ordinary multiset rewriting, with the propagation rule*

$$\mathsf{val}(i,b) \otimes \mathsf{pend}(j,i) \otimes \mathsf{rem}(j,k+1,p) \multimap \mathsf{val}(i,b) \otimes \mathsf{rem}(j,k,p \oplus b),$$

and Solve producing an ordinary $\mathsf{val}(i,p)$ *token, while edge tokens remain inert. This second correspondence holds on the images of admissible contexts.*

Proof. For the first correspondence, merging is injective on the specified contexts: the distinct predicate symbols edge and val recover the original partition. Neither rule examines a persistent-zone label or a preorder comparison. Propagate reads precisely the same value fact and rewrites precisely the same two linear atoms before and after merging. Solve consumes precisely the same three linear atoms and adds the same value and log. Hence a transition in either system has a matching transition with the same event label, and merging its endpoint gives the other endpoint. There are only these two rules, so this proves both directions of the strong bisimulation.

For ordinary rewriting, map each persistent edge and value fact to one ordinary token and retain the linear multiset. Distinct atom predicates make this map injective on admissible contexts. Propagate consumes the unique value token and immediately recreates it unchanged; thus its net effect is identical to persistent lookup. Solve introduces exactly the one new value token, while the consumed unknown token and admissibility ensure that no old value for that variable exists. The remaining consumed and produced atoms coincide rule by rule, and neither system modifies an edge token. Every applicable rule in either image therefore has the matching rule and endpoint in the other image, with the same event label. Invariant preservation ensures that subsequent contexts stay in these images. □

Thus differentiated persistent zones supply a disciplined presentation of access patterns; this particular specification does not establish their necessity or an increase in operational expressiveness. The theorem is about the two stated rewrite systems, not an equivalence between full logical calculi. In particular, it does not establish an adequacy theorem for a focused calculus of the kind studied in [And92, NM09]. The mathematical contribution used by certification is Theorem 6.10: exact trace correspondence despite delayed propagation, together with explicit invariants and a unique propagation normal form.

7. Lean mechanization and implementation boundary

The artifact uses Lean 4.14.0 with Mathlib pinned by a checked-in Lake manifest (Mathlib commit `4bbdccd9c5f8`). The library is a mechanization of finite operational states, certificate conditions and an asynchronous event system. It is not a formalization of a SELL sequent calculus, a verified Python compiler or a proof of the measured Python program's execution time.

Table 1: Mechanized results and their principal declarations. The accompanying verification report records full statements, build commands and axiom-dependency output.

Property	Lean declaration(s)
Forced assignments preserve all completions	`forced_value`, `trace_extends`
Replay matches the inductive trace relation	`replayFull_eq_some_iff`
Left-kernel separator soundness and existence	`lem_unsat_witness_sound`, `lem_unsat_witness_complete`
Accepted success and failure certificates	`thm_sound_success`, `thm_sound_failure`
Residual and empty-trace certificate completeness	`residual_unsat_iff_witness`, `thm_complete_failure`, `thm_complete_success`
Real-valued BEC likelihood maximization	`bec_completion_maximizes`, `thm_ml`
Invariant and local operational correspondence	`propagate_wf`, `solve_wf`, `solve_valid`
Propagation normalization and order independence	`propagation_normalization`, `propagation_confluence`, `propagate_commute`
Arbitrary-interleaving trace soundness	`rewstar_sound`, `adequacy_soundness`
Logged trace completeness and same-trace equivalence	`adequacy_completeness`, `thm_adequacy_complete`

Representations. Bits are `ZMod 2`, coordinates are finite types, and an observation is a function into optional bits. The formal `PeelingState` stores the partial assignment K; its unknown neighbors, degrees and syndromes are derived functions. This gives coherence by construction. The Python implementation instead maintains cached arrays and adjacency lists. The Lean update lemmas prove the mathematical degree and syndrome equations, but there is no verified refinement of those Python caches.

A formal asynchronous context stores K, finite pending sets Q_j and an ordered event observer. Its residual fields are derived from (K, Q), and its well-formedness condition is $\mathcal{N}(j) \cap U \subseteq Q_j \subseteq \mathcal{N}(j)$. Thus exact multiplicities of the canonical tokens are enforced by representation. The relation to the raw multisets and their exact multiplicity invariant is proved in Section 6; the Lean development operates on the canonical representation rather than mechanizing a quotient of all malformed raw multisets. The log field records chronological events; its multiset image represents the manuscript's unordered log atoms.

The relation `RewStar H C T D` has the event list T as an explicit index. Its soundness theorem accepts arbitrary interleavings of Propagate and Solve from a well-formed context, including endpoints with unpropagated known incidences. Completeness constructs a derivation for the very same prescribed T, ending at `encodeLogged` of the folded operational state with the previous log followed by T. It does not assert equality with an encoding that discards the Solve events, and it does not attempt to extract ordered data by eliminating a proof-irrelevant reachability proposition.

Algebra and likelihood. The separator-existence proof uses finite-dimensional linear duality: a vector outside the image of the residual matrix is separated by a linear functional, represented by a row vector over $\mathbb{F}_2$. This is an existence theorem, not a formal verification of the Python row eliminator or its transformation matrix. The residual system has a mechanized solution-or-separator equivalence. The named checker completeness theorems construct accepted certificates using the empty trace; the stronger manuscript statement for every prescribed valid partial trace follows from its residual-bijection proof and is not presented as a single checker-completeness theorem in Lean. Success and failure soundness are proved for the executable typed checker; failure soundness uses preservation of every compatible completion by the replayed trace.

The likelihood function is real-valued. It is defined as zero on an incompatible word, and otherwise as $\varepsilon^e(1-\varepsilon)^k$, where e counts erasures and $k = n - e$ counts visible entries. Lean proves that an accepted completion maximizes this function over codewords when $0 \le \varepsilon \le 1$. This is the closed form of the coordinate-product model in Proposition 4.1; the coordinate-product factorization itself is justified there rather than by a separate Lean probability-space construction. The endpoint values are included without asserting a conditional distribution on a zero-probability observation. Neither ML theorem has the vacuous conclusion `True`.

What the formal checker guarantees. The success witness in Lean is a total function on variable indices; only its residual entries are used. The Python certificate instead serializes those entries in increasing residual-index order. The adapter pads known coordinates when translating a Python witness to the typed formal input. The formal checker receives already typed matrix, observation, trace and witness data. Consequently parsing, binary-domain rejection, serialization and conversion to those types form a separate implementation boundary. Finite differential cases exercise that boundary and compare the Lean and Python acceptance results. All 575 supplied differential cases agreed, including returned words for accepted successes (369 acceptances and 206 rejections). The run used Lean's interpreter; the native executable built on this host could not be loaded by macOS 27. These finite comparisons do not prove equivalence for all inputs.

The full verification report distinguishes source inspection, compilation, theorem statements and their transitive axiom dependencies. No project axiom or admitted proof is used to establish the listed results. Lean's ordinary foundational assumptions and its kernel, together with the pinned libraries, remain part of the formal trust base. Execution of a compiled checker additionally relies on the compiler and runtime. The raw-token representation correspondence, SELL-label erasure, global rewrite termination including Solve events, stopping-set and nullity characterizations, Gaussian-elimination implementation, sparse complexity and bit-size bounds, and experimental/statistical claims are not presented as mechanized results. Their mathematical proofs or empirical evidence are given separately.

8. Experimental methodology

The experiment evaluates a Python reference implementation, not code extracted from Lean. All data discussed here were generated by a new, prespecified campaign on 23 September 2026. The earlier manuscript's outputs remain in a separate, unchanged directory and are not used in these figures or tables. The configuration, individual matrix adjacency

lists and hashes, random seeds, per-trial outcomes, repeated timings, source snapshots and environment metadata accompany the artifact.

Graphs and observations. Both constructions have variable degree three and check degree six, so $m = n/2$. The regular construction samples uniform socket permutations and rejects nonsimple graphs. The PEG variant chooses maximum graph distance and then minimum check degree, subject to a check-capacity constraint, and restarts if a construction gets stuck. It is an explicitly specified variant of the progressive edge-growth method [HEA05]; we do not assume it samples a uniform ensemble or reproduces another implementation's finite-length performance. Each family uses $n \in \{128, 256, 512, 1024\}$ and three independently seeded matrices per size, giving 24 matrices. Each matrix is reused across the channel conditions. Matrix generation, validation, dense conversion and kernel-basis construction are measured once per matrix and excluded from per-observation decoding times.

The clean campaign uses $\varepsilon \in \{0.30, 0.40, 0.50\}$ and ten observations per matrix and erasure probability, giving 720 trials. For each trial, a uniformly sampled coefficient vector in a computed kernel basis generates a uniformly sampled codeword. Erasures are independent Bernoulli variables. Distinct random streams generate the codeword, erasures, visible flips and measurement order. Their seeds are 64-bit prefixes of SHA-256 hashes of a fixed base seed and the complete trial/stream labels; the artifact records the exact encoding and checks for collisions in the executed grid.

Corruption trials use both families, $n \in \{256, 512\}$, $\varepsilon \in \{0.40, 0.55\}$ and flip probabilities $\delta \in \{0.001, 0.01\}$, with three matrices and ten observations per condition, giving 480 trials. A flip applies only at a nonerased coordinate. We record the realized number of flips, since a positive δ does not imply any bit changed. Compatibility means existence of a codeword consistent with the supplied, possibly corrupted, observation. Incompatibility is accompanied by a checked separator.

Compared operations and timing. We measure peeling only, Gaussian elimination only, hybrid decoding, hybrid decoding with certificate checking, and independent direct word verification. All algebraic pipelines use the same integer-bitset, first-available-pivot elimination engine and row-transformation tracking. The hybrid without checking still constructs its certificate; the comparison isolates execution of the checker and does not measure a producer optimized to avoid witness construction. Peeling alone is incomplete on stopping sets. Direct verification receives an already produced word and is a verification operation, not a decoder or an infeasibility test.

Each per-observation operation includes its own required initialization on a prevalidated adjacency structure. There is one warmup per operation and three timed repetitions in deterministically randomized order. Comparator times are within-trial medians. The certified pipeline also records initialization, peeling, residual construction, elimination, witness extraction, back substitution, other producer work and checking. For its stage decomposition we select the actual repetition with median internal total time; this preserves the identity

$$T_{\text{total}} = T_{\text{decode}} + T_{\text{check}}.$$

Here T_{decode} includes construction of the certificate. The internal T_{check} also includes the producer wrapper's assertion that the checker returns its proposed word; standalone replay timings exclude that wrapper comparison. We report both $R_{\text{relative}} = T_{\text{check}}/T_{\text{decode}}$ and

$R_{\text{fraction}} = T_{\text{check}}/T_{\text{total}}$. A mean of per-trial ratios is not a ratio of mean times; the condition-summary CSV provides both. The outer comparator timer also measures call overhead, so it is not silently identified with an internal stage timer.

A separate checker experiment uses 15 additional regular matrices: three each at $n \in \{256, 512, 1024, 2048, 4096\}$, six observations per matrix, $\varepsilon = 0.40$, and five randomized timing repetitions. These 90 trials use the all-zero transmitted word to isolate checking; no recovery inference is drawn from them. Each checker is called once for validation before timing. Sparse replay, dense replay and direct checking receive the same instance and resulting completion; the two replay checkers receive the same certificate. Dense input conversion is excluded, but the dense checker scans its matrix during checking.

Memory is measured separately using `tracemalloc`: peak incremental Python-tracked allocation during a call, not resident set size or total artifact storage. One observation per matrix and clean channel condition, and one per scaling matrix, is profiled. The preexisting graph, dense matrix and certificate are outside that incremental measurement. The run used an Apple M3 Pro (11 reported logical CPUs, approximately 18 GiB RAM), macOS 27.0 arm64, Python 3.12.14, NumPy 2.3.5, pandas 2.2.3 and Matplotlib 3.10.7, with `perf_counter` as the clock. Exact package versions are pinned.

Outcomes and uncertainty. Completion means acceptance of a success certificate. Exact recovery compares its word with the sampled transmitted word. Uniqueness is $|U| - \operatorname{rank}(H_U) = 0$, conditional on compatibility. These meanings are those of Section 3. Failure-certificate acceptance is recorded separately from completion. Each displayed condition at a fixed family, size and channel parameter has three matrix realizations and ten observations per matrix. Intervals in the supplementary data use 2,000 bootstrap resamples of matrix means, retaining within-matrix dependence. The scaling fit is ordinary least squares of log mean time against log blocklength; its interval resamples the three matrix means independently within each size. These descriptive intervals have limited ensemble resolution: three matrices do not support strong distributional claims, even when many channel observations or timing repetitions are available.

9. Experimental results and interpretation

The new campaign contains 720 clean-channel trials on 24 matrices, 480 corruption trials, and 90 separate checker-scaling trials. Valid completion was obtained in 1.000 of clean trials, whereas exact recovery occurred in 0.781; 174 clean residual systems were nonunique. Among 251 inputs with at least one visible flip, 100 remained compatible. A clean observation always has the transmitted codeword as a compatible completion. The 100% observed completion rate is therefore implementation evidence, not an additional recovery theorem. The exact counts are 562 exact recoveries and 546 unique completions among the 720 clean trials. All uniquely completable clean observations were recovered exactly. The remaining exact recoveries occurred when the chosen completion happened to equal the transmitted word in a nonunique system. Figure 1 separates these events. Its fixed mixture pools the four declared sizes with equal trial counts; it is not a threshold estimate for either ensemble.

Table 2 filters exactly $\varepsilon = 0.40$ and $\delta = 0$; no other erasure probabilities enter its averages. Trace and residual columns are mean numbers of variables. The check-stage column reports the internal checker stage of the selected certified repetition; direct checking

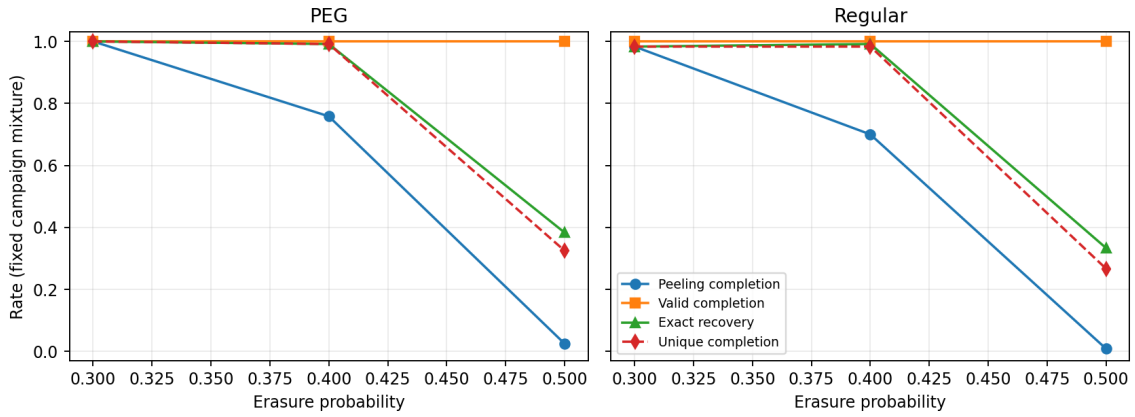


Figure 1: Clean-channel outcomes, displayed separately for each graph construction. Each point pools 120 trials: four sizes, three matrices per size and ten observations per matrix at the stated erasure probability. Valid completion, exact recovery, uniqueness and peeling completion answer different questions.

Table 2: New campaign at erasure probability 0.40: three matrices and ten trials per matrix. C/D is the internal check stage divided by decode time, averaged per trial. The check stage includes comparing checker and producer words.

Family	n	Trials	Trace	Residual	Check ms	Direct ms	Mean C/D
PEG	128	30	35.767	16.400	0.119	0.092	1.107
PEG	256	30	77.367	22.233	0.241	0.186	1.147
PEG	512	30	190.033	11.933	0.527	0.366	1.415
PEG	1024	30	389.100	22.900	1.109	0.741	1.376
Regular	128	30	33.000	18.833	0.119	0.094	0.990
Regular	256	30	84.500	15.900	0.247	0.184	1.243
Regular	512	30	174.133	32.467	0.530	0.375	1.229
Regular	1024	30	377.933	33.567	1.102	0.739	1.346

reports its separately timed median. For these 240 trials, the mean per-trial relative overhead was 1.232, whereas the mean checker fraction of total time was 0.522. The checker is therefore a substantial fraction of the measured pipeline, not negligible overhead. The corresponding ratios of mean times are 0.859 and 0.462; their difference from the mean ratios reflects variation across trials. The complete condition CSV also reports the other channel conditions.

Figure 2 uses the same algebraic engine across GE-only and hybrid pipelines. It describes this engine and finite set of graphs, without attributing a universal speed advantage to the hybrid strategy or to SELL. Table 3 separates stage means in the same fixed mixture. Detailed measurements are retained for every observation, so witness construction and checker cost remain separable from residual elimination.

At $n = 4096$, mean sparse replay took 4.546 ms, direct checking 2.966 ms, and dense replay 49.574 ms. Corresponding incremental tracked peaks were approximately 188, 72 and 5094 KiB. Thus direct checking was faster and used less additional memory than trace

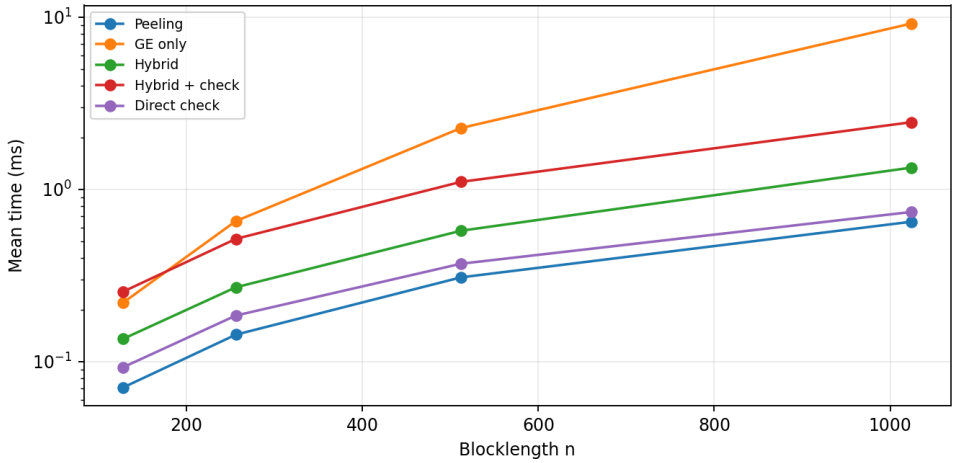


Figure 2: Mean within-trial median times at $\varepsilon = 0.40$. Each size pools 60 observations across both families, with three matrices per family and ten observations per matrix. Peeling alone may leave unknowns. Direct checking assumes an already supplied word. Its time must not be interpreted as a competing decoding latency.

Table 3: Mean certified runtime stages in milliseconds at erasure probability 0.40, pooled equally over the two named families: six matrices and 60 trials per size. Witness is final vector extraction; row-transform tracking is included in GE. Other includes parsing, output reconstruction and timing overhead; Check includes comparing checker and producer words.

n	Init	Peel	Residual	GE	Witness	Backsub	Other	Check
128	0.0134	0.0137	0.0113	0.0494	0.0001	0.0026	0.0440	0.1187
256	0.0266	0.0310	0.0200	0.1019	0.0001	0.0029	0.0872	0.2441
512	0.0576	0.0774	0.0380	0.2269	0.0001	0.0035	0.1714	0.5285
1024	0.1229	0.1867	0.0726	0.6213	0.0001	0.0047	0.3375	1.1054

replay. Replay adds a valid operational explanation; both establish the same final parity/channel property. The fitted time exponents were 1.046 (95% matrix-bootstrap interval $[1.033, 1.062]$) for sparse replay, 1.006 ($[1.003, 1.008]$) for direct checking, and 1.583 ($[1.555, 1.617]$) for dense replay. The observed sparse exponent is slightly above one; we do not force a linear fit or treat its interval as a proof of asymptotic complexity. The abstract sparse bound in Section 5 instead follows from counting incidence visits in the stated model. Dense scans and Python/runtime effects prevent identifying every measured implementation with that model.

Of 480 corruption trials, 229 realized no flip. Of the 251 with at least one flip, 151 were incompatible and yielded accepted failure certificates; 100 remained compatible and yielded accepted success certificates. In this fixed campaign, incompatibility exposed $151/251 \approx 60.2\%$ of actually corrupted inputs. The remaining $100/251 \approx 39.8\%$ cannot be rejected merely for lacking a completion. Figure 4 uses unconditional trial fractions, so its bars must

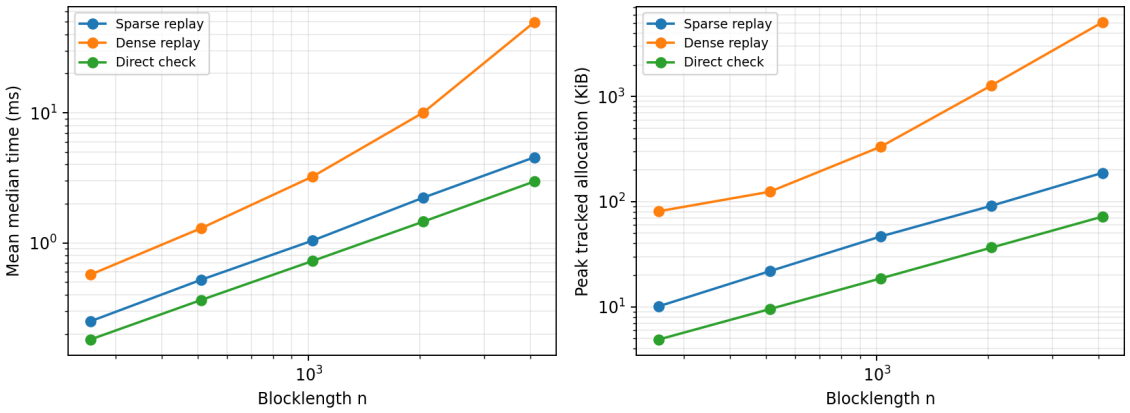


Figure 3: Independent checker scaling experiment. Time points average 18 within-trial medians (three matrices and six observations); allocation points average three separately profiled calls, one per matrix. Both axes of each panel are logarithmic. Allocations exclude preexisting inputs. The dense and sparse replay implementations check the same certificates.

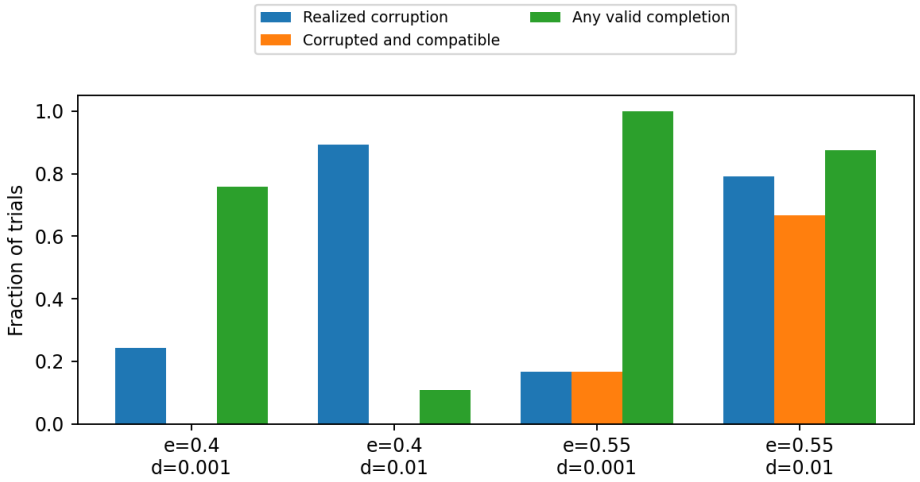


Figure 4: Visible-bit corruption: each group has 120 trials, pooling two families, two sizes, three matrices and ten observations per matrix. All bars use that same denominator. "Corrupted and compatible" is not conditional on realized corruption; it is a subset of the "Realized corruption" bar.

not be confused with these conditional rates. This is algebraic compatibility detection, not a guarantee against every BEC violation.

Reproducibility and negative evidence. The runner checks the complete declared trial grid, preserves all outcomes, checks matrix fingerprints and records raw-data hashes. An analysis-stage serialization error and a missing table-rendering dependency were repaired

after all measurements had finished; the complete original measurements were retained and only analysis was rerun. Both the executed measurement source and corrected analysis source are identifiable in the artifact. Tables and figures are generated from the CSVs by the script and reproducibility notebook. Finite adversarial tests include malformed domains and certificate lengths, altered values and indices, invalid orderings, and valid partial or independently commuting traces. Their observed acceptance or rejection is test evidence, not a theorem about the Python runtime. The direct-check advantage, nonunique completions, compatible corruption and finite-size superlinear fits are retained rather than filtered from the scientific conclusions.

10. Related work

Gallager introduced LDPC codes [Gal62]; iterative erasure coding and stopping-set analysis are developed in [LMSS01, DPT$^+$02]. Burshtein and Miller already combine iterative decoding with algebraic methods for BEC maximum-likelihood decoding [BM04]. Our contribution does not include a new hybrid decoder, stopping-set characterization, channel threshold or linear-algebraic alternative. The proofs make their role in the certificate contract explicit.

Certifying algorithms produce witnesses that can be checked separately from the producing algorithm [MMNS11]. Alkassar and coauthors connect high-level witness properties with verified checker implementations [ABMR13]. Proof-carrying code supplies a related producer/checker division for code-safety evidence [Nec97]. Our finite decoding certificates use this existing methodology. Work on linear-programming decoding is another established connection between decoding and optimization guarantees [FWK05]; its relaxation-based setting differs from checking a feasible BEC completion.

Linear logic distinguishes consumable from reusable resources [Gir87], and focusing gives structured proof search [And92]. Nigam and Miller use subexponentials in algorithmic specifications with mechanisms richer than the two persistent zones used here [NM09]. Proposition 6.12 deliberately limits our claim: the present rules admit a semantics-preserving translation to one persistent zone and to ordinary read-and-recreate multiset rewriting. The case-specific technical work is the delayed-propagation simulation and its exact trace/normalization correspondence.

Formal coding theory also predates this artifact. Affeldt, Garrigue and Saikawa formalize linear error-correcting codes in Coq and extract a verified LDPC sum-product decoder [AGS20]. Our finite-field certificate and rewriting development has a different scope; it is not the first formal treatment of LDPC decoding. A focused literature audit is included with the repository, identifying these antecedents and the limits of the novelty claim.

Recent quantum-code work further illustrates the breadth of certification. Ehatamm and coauthors develop Lean-based distance certificates through verified reductions to satisfiability [ELWT26]. Krishnamoorthy and coauthors study certificates for degenerate quantum-decoding optimality using partition-function comparisons [KGK$^+$26]. Both are 2026 preprints with different target properties: neither distance certification nor comparison of quantum error classes is the BEC trace/feasibility contract studied here. Their existence also rules out treating “Lean plus coding certificates” as a novelty claim by itself.

11. Limitations and conclusion

The architecture separates a classical hybrid producer from a checker whose accepted evidence has a precise algebraic and operational meaning. The delayed-propagation invariant is what makes arbitrary interleavings compatible with valid peeling traces. The exact endpoint includes the accumulated log, and propagation normalization supplies the connection to the fully updated operational state.

The differentiated subexponentials are a specification choice, and the paper establishes no SELLF adequacy or expressiveness separation. Trace checking validates a declared explanation of a result, not the producer's actual internal history. Direct parity and channel checking already establish the same final-word validity condition. An accepted BEC completion need not be unique or equal the transmitted word, and visible corruption may remain algebraically compatible. The supplied matrix, observation, checker and execution environment remain within the stated trust boundary.

The experimental conclusions are restricted to the recorded graph constructions, finite sizes and host environment. Three independent matrices per condition give limited information about ensemble variation, and timing fits are descriptive rather than asymptotic proofs. No universal speed advantage, universal corruption detection, verified Python extraction or fully mechanized logical calculus follows from these results. The remaining artifact and publication limitations are stated alongside the mechanization and experimental evidence.

References

[ABMR13] Eyad Alkassar, Sascha Böhme, Kurt Mehlhorn, and Christine Rizkallah. A framework for the verification of certifying computations, 2013. arXiv:1301.7462. doi:10.48550/arXiv.1301.7462.

[AGS20] Reynald Affeldt, Jacques Garrigue, and Takafumi Saikawa. A library for formalization of linear error-correcting codes. *Journal of Automated Reasoning*, 64(6):1123–1164, 2020. doi:10.1007/s10817-019-09538-8.

[And92] Jean-Marc Andreoli. Logic programming with focusing proofs in linear logic. *Journal of Logic and Computation*, 2(3):297–347, 1992. doi:10.1093/logcom/2.3.297.

[BM04] David Burshtein and Gadi Miller. An efficient maximum-likelihood decoding of LDPC codes over the binary erasure channel. *IEEE Transactions on Information Theory*, 50(11):2837–2844, 2004. doi:10.1109/TIT.2004.836694.

[DPT+02] Changyan Di, David Proietti, I. Emre Telatar, Thomas J. Richardson, and Rüdiger L. Urbanke. Finite-length analysis of low-density parity-check codes on the binary erasure channel. *IEEE Transactions on Information Theory*, 48(6):1570–1579, 2002. doi:10.1109/TIT.2002.1003839.

[ELWT26] Mattias Ehatamm, Yi Lee, Xiaodi Wu, and Runzhou Tao. End-to-end formalization of quantum error correction, 2026. arXiv preprint 2605.16523, version 1. URL: https://arxiv.org/abs/2605.16523, doi:10.48550/arXiv.2605.16523.

[FWK05] Jon Feldman, Martin J. Wainwright, and David R. Karger. Using linear programming to decode binary linear codes. *IEEE Transactions on Information Theory*, 51(3):954–972, 2005. doi:10.1109/TIT.2004.842696.

[Gal62] Robert G. Gallager. Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962. doi:10.1109/TIT.1962.1057683.

[Gir87] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–102, 1987. doi:10.1016/0304-3975(87)90045-4.

[HEA05] Xiao-Yu Hu, Evangelos Eleftheriou, and Dieter-Michael Arnold. Regular and irregular progressive edge-growth tanner graphs. *IEEE Transactions on Information Theory*, 51(1):386–398, 2005. doi:10.1109/TIT.2004.839541.

[KGK+26] Ragavi Krishnamoorthy, Florian Gerhardt, Johannes Knaute, Thomas Klir, Stefan Raimund Maschek, Erik Schulze, Tomislav Maras, Alexander Dotterweich, Loong Kuan Lee, Christian Bauckhage, and Nico Piatkowski. Certified decoding of quantum LDPC codes, 2026. arXiv preprint 2608.25545, version 1. URL: https://arxiv.org/abs/2608.25545, doi:10.48550/arXiv.2608.25545.

[LMSS01] Michael G. Luby, Michael Mitzenmacher, M. Amin Shokrollahi, and Daniel A. Spielman. Efficient erasure correcting codes. *IEEE Transactions on Information Theory*, 47(2):569–584, 2001. doi:10.1109/18.910575.

[MMNS11] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi:10.1016/j.cosrev.2010.09.009.

[Nec97] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997. doi:10.1145/263699.263712.

[NM09] Vivek Nigam and Dale Miller. Algorithmic specifications in linear logic with subexponentials. In *Proceedings of the 11th ACM SIGPLAN Conference on Principles and Practice of Declarative Programming*, pages 129–140. ACM, 2009. doi:10.1145/1599410.1599427.

[RU08] Tom Richardson and Rüdiger Urbanke. *Modern Coding Theory*. Cambridge University Press, 2008. doi:10.1017/CBO9780511791338.

This work is licensed under the Creative Commons Attribution License. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, 171 Second St, Suite 300, San Francisco, CA 94105, USA, or Eisenacher Strasse 2, 10777 Berlin, Germany